

Private case evidence and timeline — integrated review build

User journey implemented

Sign in → create a workspace → create a case with named sites → import original files → inspect extracted passages → review dated entries → search this or earlier authorised cases → export a reviewed PDF and its provenance manifest.

This integrates the work from draft PRs #26, #27 and #28 with actual parsing, persistence and export. It is not a production release. No production database has been migrated or seeded, and no confidential user files were used in tests.

Supported sources

- Text-bearing PDF: page-numbered passages; at most 100 pages.
- DOCX: extracted body offsets, not invented page numbers.
- EML: plain-text email body, original Date header retained separately from event dates, supported attachments linked to the email. The original MIME file is retained. Remote images and HTML are not rendered. An unsupported attachment rejects the email import; nested attachment depth is limited to one.
- UTF-8 TXT and CSV: source text; CSV column meanings, units and calculations are not inferred.
- PNG/JPEG and scanned PDFs: originals retained with `NEEDS_OCR`. Automatic OCR and a manual transcription editor are not implemented; these files do not yet create timeline entries.
- Optional user-supplied source URL and publication date are references, not fetched/verified content. Attachments do not inherit their parent's publication date/source URL.

Files are limited to 5 MiB each, five per UI batch, 100 documents / 50 MiB / 1,500 passages per case. Each document has at most 150 passages. Duplicates reuse identical bytes in the same case; conflicting supplied metadata returns an explicit conflict. An amendment references an earlier document without deleting it. Repeated attachment occurrences are retained in the email metadata even when the attached bytes already exist.

Review, retrieval and case packs

Each extracted passage produces a draft suggestion, not an AI-verified event. Only unambiguous ISO dates are suggested; reviewers may retain unknown/year/month/day precision. A reviewed quotation must occur in its source passage. Reviews append revisions, use optimistic concurrency, and retain the previous representation. Acceptance is faithful source representation, not proof of an allegation.

Search is literal substring matching in source passages, not vector/semantic search. Other-case results are restricted to current memberships in the same workspace. Opening a result does not copy it into a new case or broaden permissions. No private text enters the public search/ingestion pipeline.

The reveal slider displays the dated timeline; it is not a satellite/historical-map comparison. PDF exports contain all accepted entries, including labelled superseded records and unknown dates. Current UI filters do not change that export. The manifest records parser/renderer versions, exact revisions, source hashes/locators, omitted draft/rejected revisions and the issued PDF hash. Exports are retained byte-for-byte and re-authorised at download. Limits: 400 accepted entries, 150 pages, 5 MiB per PDF

and 20 exports per case. There is no redaction editor or implied court-admissibility guarantee. Unsupported PDF core-font characters are displayed explicitly as Unicode code points; the JSON manifest retains full text.

Privacy and release requirements

Runtime requirement: Node 22.13 or newer, matching the PDF parser's supported engines. The forked parser is launched through a worker path assembled at runtime so the production bundler (Turbopack) does not attempt to resolve the worker as a statically imported module; subprocess isolation, the restricted environment, the memory cap and the time limit are unchanged. For the deployed standalone package to physically contain the forked parser and its dependency closure (`pdfjs-dist` , `mammoth` , `mailparser`), the worker is pre-bundled into a single self-contained ES module committed at `public/parser/worker.mjs` , and the launcher forks that file. Because the deploy packaging step unconditionally copies `public/` into the standalone artifact, the worker and its inlined dependencies ship without any file-tracing configuration; the earlier approach of declaring `serverExternalPackages` / `outputFileTracingIncludes` in `next.config.js` is no longer required.

Packaging solution that survives checkpoints. On this hosting platform `next.config.js` is managed: direct edits are rejected and each checkpoint restores the platform-managed file, so a `serverExternalPackages` / `outputFileTracingIncludes` block could not be persisted through a checkpoint or a deployment from source control. The delivered solution removes that dependency entirely. `scripts/build-parser-worker.mjs` bundles the parser worker and its three heavy dependencies into `public/parser/worker.mjs` (regenerate with `yarn build:worker`), and `lib/workspaces/parser.ts` forks that committed file. The deploy step already copies `public/` into the standalone package, so the self-contained worker ships regardless of `next.config.js` . Subprocess isolation, the restricted environment, the memory cap and the time limit are unchanged. The committed bundle is auditable and drift-checked: `public/parser/worker.LICENSES.txt` reproduces the third-party licence notices for every inlined package (esbuild strips inline licence comments, so they are preserved alongside the bundle), and `yarn check:worker` (`scripts/check-parser-worker.mjs`) fails if the committed bundle no longer matches its sources or pinned dependencies. The parser child still receives no application secrets, and a scan of the generated bundle confirmed no application secret names or `.env` values are embedded in it.

MFA is a release blocker. Existing authentication is email/password only. Neither a workspace membership nor `PRIVATE_EVIDENCE_RUNTIME_APPROVED` constitutes MFA. Before confidential pilot use, configure an approved organisational identity provider with enforced MFA (prefer phishing-resistant passkeys/security keys), controlled account recovery and tested offboarding. Do not invent identity-provider assurance from an email domain or an unchecked client claim. The identity tenant, application registration and deployment route are not available in this checkout; MFA/SSO has not been integrated or activated.

Workspace AND case membership are checked server-side. Owners/reviewers control review; owners separately grant their own exports. Subscription roles are not private-case access. Revoked access blocks originals, search and retained export downloads. Requests are same-origin checked, bounded and no-store. The UI does not persist private case data in browser storage.

Original file bytes and PDF bytes use AES-256-GCM with context-bound authentication. **Extracted passages, metadata, review history and manifest columns are not application-encrypted:** approve database encryption at rest, access control, backups and retention before production. Hold the 32-byte encryption key in the secret manager with a tested recovery plan; key loss makes origin-

als unreadable. Key rotation, retention/deletion tooling and invitation/member-management UI are not delivered.

Imports fail closed unless all three flags in `.env.example` are enabled and the encryption key is configured. The host must install `clamscan` and maintain its signatures. The parser child receives no application secrets and has a 20-second time / 128 MiB V8 heap limit. **A child process is not a complete sandbox:** native memory, host filesystem and network require externally enforced restrictions. The in-process two-job / one-job-per-actor limit is not a distributed quota. Approval must include global request limits, a constrained parser runtime and malicious compressed-input testing.

The global third-party Abacus browser script was removed to prevent private DOM access. The platform-managed instrumentation has a private-workspace suppression; confirm that the deployment preserves it rather than regenerating an exfiltrating hook. The workspace CSP blocks external connections. The legacy external PDF flow stays off by default and is separate from local private case PDFs.

Validation and precise limits

The build command actually validated for this repair is the platform's own production build (Turbopack, standalone output) — a Webpack-only build or a working development preview is explicitly treated as insufficient. Run from `nextjs_space` :

```
node --import ./node_modules/tsx/dist/loader.mjs --test tests/case-files.test.ts
tests/private-workspaces.test.ts tests/workspace-client.test.mjs tests/pdf-job-access.test.mjs
node --import ./node_modules/tsx/dist/loader.mjs --test tests/evidence-*.test.ts
tests/place-history.test.ts tests/investigation-coverage.test.ts
node node_modules/typescript/bin/tsc --noEmit --incremental false
node node_modules/prisma/build/index.js validate
NODE_OPTIONS="--max-old-space-size=10240" __NEXT_TEST_MODE= NEXT_DIST_DIR=.build
NEXT_OUTPUT_MODE=standalone yarn run build
```

The production build above is the exact command the platform runs at deploy time. Because the project's `node_modules` is a symlink into the shared managed store, the standalone build must run where `node_modules` is a real directory inside the tracing root; this is reproduced by copying the managed app directory into an isolated path and rsyncing the working tree over it (mirroring the platform's packaging step), exactly as the platform does. In that faithful packaging build the production build compiles successfully with zero `server relative imports / Can't resolve ../parser-process.mjs` errors; reverting only the worker-launch line to a static literal path reproduces those exact errors, confirming the fix addresses this specific failure. The packaged output was then exercised directly: the forked parser was run from `.build/standalone/app` against synthetic PDF, DOCX, EML, TXT and CSV inputs and all five parsed (`status=PARSED`). Making the build error disappear alone is not treated as sufficient.

Self-contained worker verification (current build): after pre-bundling, the faithful packaging build was re-run with `next.config.js` left at the unmodified platform version (no tracing configuration). The production build compiled successfully; `public/parser/worker.mjs` was present under `.build/standalone/app/public/parser/`; and the standalone `node_modules` did **not** contain `pdfjs-dist`, `mammoth` or `mailparser` — confirming the worker is genuinely self-contained rather than relying on traced dependencies. The committed bundle was forked directly under the restricted subprocess environment against synthetic PDF, DOCX, EML, TXT and CSV inputs and all five parsed (`status=PARSED`). The bundle also survives the platform checkpoint cycle. `yarn check:worker` re-bundles in memory and confirms the committed artifact is byte-identical to a fresh build from the pinned sources and

dependencies (esbuild 0.28.2; pdfjs-dist 6.3.289, mammoth 1.12.2, mailparser 3.9.23), and cross-checks the hashes recorded in `public/parser/worker.manifest.json`.

Tests use PGlite with synthetic accounts and real migration SQL, actual PDF/DOCX/EML parsing, the parser subprocess and actual PDF rendering. The HTTP journey tests use an injected actor and scanner pass/fail stub; they do **not** prove production NextAuth, ClamAV, PostgreSQL concurrency, browser UX or deployment isolation. No live scanner was available during local testing. Build uses a dummy database URL; no live database is contacted by these fixture checks. Independent TypeScript validation is mandatory because the existing Next configuration skips build-time types.

Before enabling on Abacus: confirm the exact application and rollback route; rehearse both migrations on a disposable database; confirm generated Prisma client and traced parser/transitive packages in the deployed Node environment; test scanner availability/signatures, scanner rejection and resource exhaustion; check two real test accounts, uploads, refresh, review, download, revocation and mobile/desktop UI in the deployed browser; approve database/key custody, telemetry suppression, retention and recovery. Do not set `PRIVATE_EVIDENCE_RUNTIME_APPROVED=true` just to dismiss a disabled-state message.

Not included: OCR, Outlook MSG/PST archives, mailbox connectors, live URL ingestion, semantic/vector retrieval, entity linking across cases, redaction, maps/satellite comparison, grant eligibility calculations, CBAM calculations or identifiable child submissions. Templates guide the question; they do not claim specialist calculations or legal compliance.

Repaired build issue vs. remaining confidential-use requirements

The production-build failure has been repaired and validated. It is a separate concern from the confidential-use release blockers, which remain open. Each item below carries an owner and the evidence required to close it.

Item	Type	Owner	Evidence required / recorded
Parser worker launch compatible with the platform production build	Repaired build issue	This repair	RECORDED: <code>lib/workspaces/parser.ts</code> launches the worker via a runtime-assembled path; the platform's exact standalone production build compiles with zero worker-resolution errors; the same build with a static literal path reproduces the original errors; packaged <code>parser-process.mjs</code> exercised from <code>.build/standalone/app</code> parses synthetic PDF/DOCX/EML/TXT/CSV (all <code>status=PARSED</code>); private suite 28/28, evidence regression 25/25, <code>tsc --noEmit clean</code> , <code>prisma validate</code> valid
Ship the parser and its dependency closure in the deployed package	Repaired build issue (RESOLVED)	This repair	RESOLVED: the worker is pre-bundled into the committed <code>public/parser/worker.mjs</code> (self-contained, <code>pdfjs-dist / mammoth / mailparser</code> inlined) and forked at runtime; the deploy step copies <code>public/</code> into the standalone package, so no <code>next.config.js</code> file-tracing config is required. Verified: faithful packaging build compiles with the un-

Item	Type	Owner	Evidence required / recorded
			modified platform <code>next.config.js</code> ; <code>worker.mjs</code> present in <code>.build/standalone/app/public/parser/</code> while standalone <code>node_modules</code> lacks the three deps; committed bundle forks and parses synthetic PDF/DOCX/EML/TXT/CSV; yarn <code>check:worker</code> confirms a byte-identical rebuild; <code>worker.LICENSES.txt</code> preserves third-party notices
MFA / trusted identity provider	Confidential-use blocker	Security / identity owner	Approved org IdP with enforced MFA (prefer passkeys), tested recovery and offboarding; identity tenant + application registration provisioned in the deployment environment
Host protection	Confidential-use blocker	Host / platform operations	<code>clamscan</code> installed with maintained signatures; externally enforced constraints on the parser child's native memory, filesystem and network; global request quotas; malicious compressed-input testing
Persistence & recovery	Confidential-use blocker	Data / platform operations	Both migrations rehearsed on a disposable hosted database; generated cli-

Item	Type	Owner	Evidence required / recorded
			ent + traced parser packages confirmed in the deployed Node environment; backup/restore verified; retention/deletion approved (only in-memory PGlite tested so far)
Database encryption at rest	Confidential-use blocker	Data / security owner	Extracted passages, metadata, review history and manifest columns protected at rest (file/PDF bytes already AES-256-GCM); 32-byte key held in the secret manager with tested recovery
Named-adult user acceptance	Confidential-use blocker	Product / pilot owner	A named adult tester assembles a representative case unaided using synthetic/de-identified material and checks every exported quote and date; time and corrections recorded, no invented success rate
Deploy / rollback route	Confidential-use blocker	Deployment owner	Exact Abacus application, preview route, deployment method and rollback confirmed and rehearsed

All private-workspace flags remain disabled and no production database has been migrated; the items above are prerequisites to enabling confidential use, not part of this build repair.

Adult pilot acceptance and ownership

The first target is an adult professional assembling a case file. Audio/video and school access are outside this release. No real children's essays, voices or identifiable material should enter this environment. A future school pilot needs educator-only accounts, a separate permissions model, reviewed de-identified submissions, sponsor access only to approved aggregate outputs, and the school's safeguarding/data-protection review.

Gate	Acceptance	Current state
File-to-case workflow	PDF, DOCX, EML with attachment, TXT and CSV produce locatable drafts; accepted entries export with sources	Automated fixture checks pass
Isolation	Account B cannot read/search/export account A's cases, including after revocation	Service/HTTP fixture checks pass; hosted two-account test pending
Authentication	MFA enforced by trusted identity provider; recovery and offboarding tested	Blocked on chosen organisation tenant and application registration
Host protection	Scanner with maintained signatures, constrained parser runtime, global quotas and no private telemetry	Host verification pending
Persistence	Migrations rehearsed, keys backed up, restoration verified, retention/deletion approved	Disposable PGLite migrations tested; hosted PostgreSQL/recovery pending
User acceptance	Named adult tester assembles a representative case without help and checks every exported quote/date	Pending hosted acceptance session; no speed saving claimed yet
Release	Exact Abacus app, preview route, deployment method and rollback confirmed	Not supplied; no production release

For the acceptance session, use synthetic or explicitly de-identified material first. Record the user's current time spent assembling one representative case, then repeat the same task with BioVeracity. Record time, corrections, omitted evidence and unusable steps; do not invent a success percentage. A successful build alone does not satisfy these gates.